

Semana 25 - Aula 1

Tópico Principal da Aula: Implementação e manutenção de APIs seguras

Subtítulo/Tema Específico: Implementação de OAuth

Código da aula: [SIS] ANO2C2B4S25A1

Objetivos da Aula:

- Implementar OAuth para autenticação e autorização.
- Desenvolva uma curiosidade ao explorar OAuth.

Recursos Adicionais:

- Caderno para anotações;
- Computador com internet;
- Roteiro de atividade prática.

Exposição do Conteúdo:

Referência do Slide: Slide 6 - Descomplicando o OAuth: autorização segura para APIs

- **Definição:** OAuth (Open Authorization) é um padrão aberto para delegação de acesso. Ele permite que um usuário conceda a um aplicativo de terceiros acesso a seus recursos armazenados em outro serviço, sem precisar compartilhar suas credenciais (como nome de usuário e senha).
- **Aprofundamento/Complemento:** O OAuth 2.0 é o framework de autorização padrão da indústria. Ele funciona com base em "tokens" de acesso. Em vez de a senha do usuário ser fornecida a uma aplicação, o serviço que detém os dados do usuário (como Google, Facebook, etc.) emite um token de acesso para a aplicação. Esse token tem um escopo limitado (define o que a aplicação pode fazer) e um tempo de vida limitado, aumentando a segurança.
- **Exemplo Prático:** Quando você usa a opção "Entrar com o Google" em um site ou aplicativo. Você não digita sua senha do Google no site, mas é redirecionado para uma página de login do Google. Após a autenticação, a pergunta do Google se você permite que o site acesse certas informações (como seu nome e e-mail). Ao concordar, o Google envia um token de autorização para o site, permitindo o acesso limitado, sem nunca expor sua senha.
- **Link do vídeo:** O que é OAuth 2.0? https://www.youtube.com/watch?v=g_j-vde4b_A

Referência do Slide: Slides 7 a 12 - Fluxos de Autorização e Boas Práticas

- **Definição:** A escolha do fluxo de solicitação no OAuth é crucial e depende do tipo de aplicação (web, mobile, aplicativo de página única). O "Fluxo de Código de Autorização" é um dos mais seguros e comuns. Neste fluxo, a aplicação troca um código de autorização por um token de acesso diretamente com o servidor de autorização, o que impede a exposição do token ao cliente.

- **Aprofundamento/Complemento:** Outros fluxos incluem o "Implicit Flow" (menos seguro, para aplicações de página única que não podem salvar um segredo do cliente) e o "Client Credentials Flow" (para comunicação máquina-a-máquina). A configuração correta envolve:
 - **Registro da Aplicação:** Cadastrar a aplicação no provedor de autorização para obter um `client_id` e `client_secret`.
 - **URLs de redirecionamento:** Defina URLs seguras e específicas para onde o usuário será enviado após autorização.
 - **Escopos:** Limitar os escopos de acesso, solicitando apenas as tarefas necessárias..
 - **Gestão de Tokens:** Utilizar tokens com tempo de expiração curto e mecanismos de renovação (refresh tokens) para minimizar riscos.
- **Exemplo Prático:** Uma aplicação web de gerenciamento de tarefas que quer acessar os eventos do Google Calendar de um usuário. Ao registrar a aplicação, o desenvolvedor define o escopo como `calendar.readonly`. Quando o usuário autoriza, o aplicativo recebe um `authorization code`. O servidor da aplicação, usando seu `client_id` e `client_secret`, troque esse código por um `access token` com o servidor do Google. Este token é então usado para ler os eventos do calendário, mas não pode modificá-los. Se o token vazar, ele expirará em pouco tempo.
- **Link do vídeo:** OAuth 2.0 e OpenID Connect (em inglês simples) - <https://www.youtube.com/watch?v=996OiexHze0>

Semana 25 - Aula 2

Tópico Principal da Aula: Implementação e manutenção de APIs seguras

Subtítulo/Tema Específico: Utilização de JSON Web Token (JWT)

Código da aula: [SIS] ANO2C2B4S25A2

Objetivos da Aula:

- Utilização JWT para autenticação segura.
- Implementar OAuth para autenticação e autorização.
- Desenvolva uma curiosidade ao explorar OAuth.

Recursos Adicionais:

- Caderno para anotações;
- Recurso audiovisual para exibição de vídeos e imagens.

Exposição do Conteúdo:

Referência do Slide: Slides 8 a 12 - Conceito e Estrutura de um JWT.

- **Definição:** JWT (JSON Web Token) é um padrão aberto (RFC 7519) que define uma forma compacta e auto suficiente para transmitir informações de forma segura entre partes como um objeto JSON. Por serem assinados digitalmente, as informações contidas podem ser verificadas.

- **Aprofundamento/Complemento:** Um JWT é composto por três partes separadas por pontos (.):
 - **Header (Cabeçalho):** Contém metadados sobre o token, como o tipo (typ) e o algoritmo de assinatura (alg).
 - **Payload (Carga Útil):** Contém as "claims" (declarações), que são informações sobre o usuário e dados adicionais.
 - **Assinatura (Assinatura):** É usada para verificar a atualização e integridade do token. É criado usando o cabeçalho, o payload, uma chave secreta e o algoritmo especificado.
- **Exemplo Prático:**
 - **Cabeçalho:** { "alg": "HS256", "typ": "JWT" }
 - **Carga útil:** { "sub": "1234567890", "name": "João Silva", "admin": true }
 - **Assinatura:** HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret_key)
- **Link do Vídeo:** O que é JWT? (Token Web JSON)
- https://www.youtube.com/watch?v=C-p_gR454S8

Referência do Slide: Slides 13 a 20 - Vantagens e Implementação do JWT

- **Definição:** As principais vantagens do JWT são:
 1. **Autenticação Stateless:** Cada requisição contém toda a informação necessária, sem depender de sessões no servidor.
 2. **Escalabilidade:** Facilita a expansão da aplicação, pois a autenticação não depende de estado no servidor.
 3. **Integridade e Autenticidade:** A assinatura digital previne adulterações.
 4. **Interoperabilidade:** Funciona em diferentes plataformas e linguagens.
- **Aprofundamento/Complemento:** A implementação do JWT segue três fases:
 1. **Geração:** Após o login, o servidor cria um JWT assinado.
 2. **Envio:** O token é enviado ao cliente para armazenamento.
 3. **Validação:** A cada requisição, o cliente envia o JWT no cabeçalho Authorization e o servidor verifica a assinatura.
- **Exemplo Prático (Node.js):**
- JavaScript

// Geração do Token

```
const jwt = require('jsonwebtoken');
const payload = { sub: user.id, name: user.name }; // [cite: 256]
const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '1h' }); // [cite: 257]
```

// Middleware de Validação

```
function authenticateToken(req, res, next) {
  const token = req.headers['authorization']?.split(' ')[1]; // [cite: 260]
  if (!token) return res.sendStatus(401); // [cite: 260]
```

```
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => { // [cite: 261]
    if (err) return res.sendStatus(403); // [cite: 261]
    req.user = user; // [cite: 261]
```

```
next(); // [cite: 261]
});
}
```

-
-
- **Link de vídeo:** API com autenticação JWT em Node.js - https://www.youtube.com/watch?v=f55OMg_C53s

Semana 25 - Aula 3

Tópico Principal da Aula: Implementação e manutenção de APIs seguras

Subtítulo/Tema Específico: Auditoria e logs de segurança

Código da aula: [SIS] ANO2C2B4S25A3

Objetivos da Aula:

- Implementar auditorias e registros para monitorar segurança.
- Implementar OAuth para autenticação e autorização.
- Desenvolva uma curiosidade ao explorar OAuth.

Recursos Adicionais:

- Caderno para anotações;
- Computador com internet;
- Roteiro de atividade prática.

Exposição do Conteúdo:

Referência do Slide: Slides 6 e 7 - Rastreando ameaças: auditorias e logs para APIs seguras

- **Definição:** Auditorias de segurança e registros são processos para monitorar, detectar e responder a incidentes de segurança. Logs são registros de eventos, e os auditórios são o processo de análise para investigar atividades suspeitas.
- **Aprofundamento/Complemento:** Uma melhoria suave envolve:
 - **Centralização de Logs:** Use ferramentas como ELK Stack ou Splunk para agregar logs em um único local.
 - **Configuração Adequada:** Registrar eventos críticos como esforço de login, acessos a dados sensíveis e erros
 - **Proteção e Retenção:** Protege os logs com criptografia e define políticas de retenção .
- **Exemplo Prático:** Um sistema registra cada tentativa de acesso administrativo. Se um invasor tentar um ataque de força brutal, os logs serão registrados múltiplos eventos de "falha de login" do mesmo IP, gerando um alerta para a equipe de segurança.
- **Link do Vídeo:** O que são Logs? - <https://www.youtube.com/watch?v=BVqFpbFiV34>

Referência do Slide: Slide 11 - Análise e Comunicação

- **Definição:** A análise contínua dos logs é fundamental para identificar padrões anômalos e reagir a incidentes. A comunicação eficaz entre as equipes de segurança e TI é crucial para uma resposta rápida.
- **Aprofundamento/Complemento:**
 - **Métodos de Análise:** Usar dashboards, filtros e alertas automáticos para identificar eventos críticos.
 - **Canais de Comunicação:** Criar canais dedicados (Slack, Teams) para notificar incidentes .
 - **Reuniões Periódicas:** Realizar reuniões para revisar incidentes e alinhar estratégias de melhoria .
- **Exemplo Prático:** A equipe de segurança percebe um aumento de erros 403 (Forbidden) em um painel. Eles usam um canal do Slack para reunir desenvolvedores e a equipe de DevOps, analisar os logs, identificar que uma regra de firewall está bloqueando requisições e resolver o problema em minutos.
- **Link do Vídeo:** O que é SIEM? Gerenciamento e Correlação de Eventos de Segurança -https://www.youtube.com/watch?v=f_VpPGeGqGg